

منصة فرصة

Project Documentation - توثيق المشروع الكامل

مشروع توظيف رقمي متكامل لقطاع غزة

Full-Stack JavaScript — Node.js / Express / React / MongoDB

الإصدار	1.0.0
التاريخ	2026
اللغة التقنية	JavaScript (Frontend + Backend)
نوع المشروع	Web Application - Full Stack
الجمهور المستهدف	سوق العمل في قطاع غزة
الحالة	مرحلة التخطيط والتصميم

نظرة عامة على المشروع 1.

منصة فرصة هي تطبيق ويب متكامل يهدف إلى تنظيم سوق العمل الرقمي داخل قطاع غزة، من خلال توفير بيئة احترافية وأمنة تجمع أصحاب العمل والباحثين عن فرص الشغل في مكان واحد، بعيداً عن الفوضى وعدم التنظيم السائد في الجروبات على وسائل التواصل الاجتماعي.

المشكلة التي يحلها المشروع 1.1

يعاني سوق العمل في غزة من غياب منصة رقمية متخصصة، مما يجعل الباحثين عن العمل وأصحاب الفرص يلجؤون إلى مجموعات واتساب وفيسبوك التي تقتصر للتنظيم، وتُضيع الفرص وسط الفوضى. تأتي منصة فرصة لتحل هذه المشكلة من خلال:

- توحيد الفرص في مكان واحد منظم وسهل البحث
- توفير آلية تقديم رسمية بدلاً من المراسلة العشوائية
- ضمان جودة الفرص المنشورة عبر نظام الإشراف الإداري
- إتاحة فرص العمل الأونلاين والميداني في نفس المنصة

أهداف المشروع 1.2

1. بناء منصة توظيف رقمية متكاملة ومخصصة لبيئة غزة
2. تسهيل التواصل بين أصحاب العمل والباحثين عن فرص
3. توفير لوحة تحكم لكل نوع من المستخدمين
4. ضمان جودة المحتوى عبر نظام موافقة الأدمن
5. دعم رفع السيرة الذاتية والتقديم المباشر عبر الموقع
6. إرسال إشعارات فورية عند أي حدث مهم

أنواع المستخدمين والصلاحيات 2.

يتعامل الموقع مع ثلاثة أنواع رئيسية من المستخدمين، لكل منهم صلاحيات وواجهة مخصصة:

نوع المستخدم	الوصف	الصلاحيات الرئيسية
باحث عن عمل (Job Seeker)	أي شخص يبحث عن فرصة شغل أو فريланس	التقديم، حفظ CV، تصفح الفرص، رفع الفرص، استقبال إشعارات
صاحب فرصة (Employer)	أفراد أو جهات تريد نشر فرصة شغل	نشر فرص، مراجعة المتقدمين، قبول/رفض التقديمات، إدارة فرصه
أدمن (Admin)	المشرف على المنصة لضمان الجودة	مراجعة الفرص المنشورة، الموافقة أو الرفض، إدارة المستخدمين، إحصائيات

رحلة المستخدم - باحث عن عمل 2.1

7. التسجيل وإنشاء حساب
8. رفع السيرة الذاتية (PDF أو Word)
9. تصفح الفرص والفلتره حسب التخصص أو النوع
10. التقديم على الفرص المناسبة بضغطه واحدة
11. متابعة حالة التقديمات من لوحة التحكم
12. استقبال إشعار عند قبول أو رفض التقديم

رحلة صاحب الفرصة 2.2

13. Employer التسجيل وإنشاء حساب
14. نشر فرصة جديدة (وصف، متطلبات، نوع، طريقة التواصل)
15. انتظار موافقة الأدمن على الفرصة
16. استقبال التقديمات على الفرصة
17. المتقدمين وقبول أو رفض كل تقديم CVs مراجعة
18. إغلاق الفرصة عند الانتهاء

رحلة الأدمن 2.3

19. تسجيل الدخول إلى لوحة الإدارة
20. (Pending Jobs) مراجعة الفرص المعلقة
21. الموافقة على الفرصة أو رفضها مع سبب الرفض
22. إدارة المستخدمين (تعطيل، حذف، ترقية)
23. متابعة إحصائيات المنصة

3. الهيكل التقني للمشروع

Backend والـ Frontend بالكامل، مع فصل واضح بين الـ JavaScript بلغة Full-Stack يعتمد المشروع على بنية

3.1 التقنيات المستخدمة

الطبقة	التقنية	الاستخدام
Frontend	React.js + Vite	SPA واجهة المستخدم والـ
Styling	Tailwind CSS	تصميم سريع ومتجاوب
State Management	Zustand / Context API	إدارة الحالة
HTTP Client	Axios	API التواصل مع الـ
Backend	Node.js + Express.js	Server والـ REST API الـ
Database	MongoDB + Mongoose	قاعدة البيانات
Authentication	JWT (JSON Web Tokens)	نظام تسجيل الدخول
File Upload	Multer + Cloudinary	رفع الملفات والصور
Notifications	Socket.IO	الإشعارات الفورية
Email	Nodemailer	إرسال الإيميلات
Validation	Joi / express-validator	التحقق من البيانات
Password	Bcrypt.js	تشفير كلمات المرور

3.2 Backend - هيكل المجلدات

```
fursa-backend/  
├── src/  
│   ├── config/  
│   │   ├── db.js           # اتصال قاعدة البيانات  
│   │   └── cloudinary.js   # إعدادات Cloudinary  
│   ├── models/  
│   │   ├── User.model.js   # نموذج المستخدم  
│   │   ├── Job.model.js    # نموذج الفرصة  
│   │   └── Application.model.js # نموذج التقديم  
│   ├── controllers/  
│   │   ├── auth.controller.js  
│   │   ├── job.controller.js  
│   │   ├── application.controller.js  
│   │   └── admin.controller.js  
│   ├── routes/  
│   │   ├── auth.routes.js  
│   │   ├── job.routes.js  
│   │   ├── application.routes.js  
│   │   └── admin.routes.js  
│   └── middlewares/  
└──
```

```

├── auth.middleware.js # JWT التحقق من
├── role.middleware.js # التحقق من الصلاحية
├── upload.middleware.js
├── utils/
│   ├── sendEmail.js
│   └── apiResponse.js
├── app.js
├── .env
├── server.js
└── package.json

```

3.3 Frontend - هيكل المجلدات

```

fursa-frontend/
├── src/
│   ├── components/
│   │   ├── common/ # مكونات مشتركة (Navbar, Footer, Button...)
│   │   ├── jobs/ # JobCard, JobList, JobFilter
│   │   └── dashboard/ # مكونات لوحة التحكم
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Jobs.jsx
│   │   ├── JobDetail.jsx
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   └── dashboard/
│   │       ├── SeekerDashboard.jsx
│   │       ├── EmployerDashboard.jsx
│   │       └── AdminDashboard.jsx
│   │   └── PostJob.jsx
│   ├── store/ # Zustand stores
│   ├── hooks/ # Custom hooks
│   ├── services/ # API calls (axios)
│   ├── utils/
│   └── App.jsx
├── public/
├── index.html
└── package.json

```

4. تصميم قاعدة البيانات (MongoDB Schemas)

4.1 نموذج المستخدم (User Schema)

```
const UserSchema = new mongoose.Schema({
  name:      { type: String, required: true },
  email:     { type: String, required: true, unique: true },
  password:  { type: String, required: true },
  role:      { type: String, enum: ['seeker','employer','admin'], default:
'seeker' },
  phone:     { type: String },
  location:  { type: String },
  bio:       { type: String },
  cvUrl:     { type: String }, // رابط السيرة الذاتية
  savedJobs: [{ type: ObjectId, ref: 'Job' }],
  isActive:  { type: Boolean, default: true },
}, { timestamps: true });
```

4.2 نموذج الفرصة (Job Schema)

```
const JobSchema = new mongoose.Schema({
  title:      { type: String, required: true },
  description: { type: String, required: true },
  requirements: { type: String },
  type:       { type: String, enum: ['online','field','hybrid'] },
  category:   { type: String }, // تخصص: برجة، تصميم، إلخ
  contactInfo: { type: String, required: true },
  employer:   { type: ObjectId, ref: 'User', required: true },
  status:     { type: String, enum: ['pending','approved','rejected'], default:
'pending' },
  rejectionReason: { type: String },
  isOpen:     { type: Boolean, default: true },
  deadline:   { type: Date },
}, { timestamps: true });
```

4.3 نموذج التقديم (Application Schema)

```
const ApplicationSchema = new mongoose.Schema({
  job:        { type: ObjectId, ref: 'Job', required: true },
  applicant:  { type: ObjectId, ref: 'User', required: true },
  coverLetter: { type: String },
  cvUrl:      { type: String }, // خاص بهذا التقديم CV
  status:     { type: String, enum: ['pending','accepted','rejected'], default:
'pending' },
  seenByEmployer: { type: Boolean, default: false },
}, { timestamps: true });
```

5. API Endpoints توثيق

5.1 المصادقة والحسابات (Auth)

Method + Endpoint	الوصف	الصلاحيات المطلوبة
POST /api/auth/register	تسجيل مستخدم جديد	عام
POST /api/auth/login	JWT تسجيل الدخول والحصول على	عام
GET /api/auth/me	الحصول على بيانات المستخدم الحالي	مسجل دخول
PUT /api/auth/profile	تحديث بيانات الملف الشخصي	مسجل دخول
POST /api/auth/upload-cv	رفع السيرة الذاتية	Seeker

5.2 الفرص الوظيفية (Jobs)

Method + Endpoint	الوصف	الصلاحيات المطلوبة
GET /api/jobs	جلب كل الفرص المعتمدة مع فلترة وبحث	عام
GET /api/jobs/:id	جلب تفاصيل فرصة معينة	عام
POST /api/jobs	pending) نشر فرصة جديدة (تصبح	Employer
PUT /api/jobs/:id	تعديل فرصة (قبل الموافقة فقط)	Employer ((صاحبها)
DELETE /api/jobs/:id	حذف فرصة	Employer / Admin
GET /api/jobs/my-jobs	الحالي Employer جلب فرص الـ	Employer
POST /api/jobs/:id/save	حفظ / إلغاء حفظ فرصة	Seeker
GET /api/jobs/saved	جلب الفرص المحفوظة	Seeker

5.3 التقديمات (Applications)

Method + Endpoint	الوصف	الصلاحيات المطلوبة
POST /api/applications/:jobId	التقديم على فرصة	Seeker
GET /api/applications/my	Seeker جلب تقديماتي كـ	Seeker
GET	جلب المتقدمين على فرصة معينة	Employer

/api/applications/job/:jobId		
PUT /api/applications/:id/status	قبول أو رفض تقديم	Employer

5.4 لوحة الإدارة (Admin)

Method + Endpoint	الوصف	الصلاحيات المطلوبة
GET /api/admin/jobs/pending	جلب الفرص المعلقة للمراجعة	Admin
PUT /api/admin/jobs/:id/approve	الموافقة على فرصة	Admin
PUT /api/admin/jobs/:id/reject	رفض فرصة مع سبب	Admin
GET /api/admin/users	جلب كل المستخدمين	Admin
PUT /api/admin/users/:id/toggle	تفعيل/تعطيل مستخدم	Admin
GET /api/admin/stats	إحصائيات المنصة	Admin

6. الميزات التفصيلية.

6.1 نظام الفلترة والبحث

يدعم الموقع فلترة الفرص بعدة معايير متزامنة:

- البحث بالكلمات المفتاحية في العنوان والوصف
- الفلترة حسب نوع العمل: أونلاين / ميداني / هجين
- الفلترة حسب التخصص: برمجة، تصميم جرافيك، ترجمة، تدريس، إلخ
- الترتيب حسب: الأحدث، الأكثر تقديمات
- Pagination لعرض النتائج على صفحات

6.2 نظام رفع الملفات

تدفق رفع السيرة الذاتية

يُخزن الرابط في قاعدة → Cloudinary يُرسل إلى → Backend يستقبله في الـ Multer → المستخدم يرفع الملف من المتصفح عند مراجعة التقديم Employer البيانات → يُعرض للـ

- الصيغ المدعومة: PDF, DOC, DOCX
- الحجم الأقصى: 5 ميغابايت لكل ملف
- في أي وقت CV تحديث Seeker يمكن للـ
- الافتراضي CV مختلف مع كل تقديم أو استخدام الـ CV يمكن رفع

6.3 نظام الإشعارات (Socket.IO)

محتوى الإشعار	من يُرسل إليه	الحدث
تقديم جديد على فرصتك: [اسم الفرصة]	(Employer) صاحب الفرصة	تقديم جديد على فرصة
تم قبول تقديمك على: [اسم الفرصة]	(Seeker) الباحث عن العمل	قبول تقديم
عذراً، لم يتم قبول تقديمك على: [اسم الفرصة]	(Seeker) الباحث عن العمل	رفض تقديم
تمت الموافقة على فرصتك ونشرها	Employer	موافقة الأدمن على فرصة
تم رفض فرصتك، السبب: [...]	Employer	رفض الأدمن لفرصة

6.4 لوحات التحكم (Dashboards)

لوحة تحكم الباحث عن عمل

- ملخص: عدد التقديمات، المقبولة، المرفوضة، المعلقة
- جدول تقديماتي مع حالة كل تقديم
- قائمة الفرص المحفوظة
- CV إدارة الملف الشخصي والـ

لوحة تحكم صاحب الفرصة

- ملخص: عدد فرصي، المعتمدة، المعلقة، المرفوضة
- إدارة الفرص المنشورة (تعديل، حذف، إغلاق)
- مراجعة المتقدمين على كل فرصة
- المتقدمين CVs تحميل وقراءة

لوحة تحكم الأدمن

- إحصائيات: إجمالي المستخدمين، الفرص، التقديمات
- قائمة الفرص المعلقة للمراجعة
- أدوات إدارة المستخدمين
- سجل النشاط (Activity Log)

7. الأمان والحماية

مبادئ الأمان المتبعة في المشروع

طبقات متعددة من الحماية بحيث لا يكفي اختراق طبقة واحدة للوصول إلى — Defense in Depth يعتمد المشروع على مبدأ البيانات.

الهدف	الآلية	المجال
منع الوصول غير المصرح	مع انتهاء صلاحية (7 أيام) JWT	المصادقة
تشفير آمن لكلمات المرور	Bcrypt (salt rounds: 12)	كلمات المرور
منع الوصول خارج الصلاحيات	Role-based middleware لكل route	الصلاحيات
منع رفع ملفات خطيرة	فحص نوع الملف + حجمه	رفع الملفات
منع البيانات المشوهة	Joi validation لكل API	المدخلات
منع الطلبات من مصادر مجهولة	المسموحة origins قائمة بيضاء للـ	CORS
Brute Force منع هجمات	login على express-rate-limit	Rate Limiting
XSS و حماية من هجمات Clickjacking	أمنة HTTP headers	Helmet.js

خطة العمل والمراحل 8.

المرحلة الأولى: الإعداد والبنية التحتية (الأسبوع 1-2)

الأهداف

إعداد بيئة العمل، هيكل المجلدات، الاتصال بقاعدة البيانات، وإعداد المصادقة الأساسية.

24. Backend للـ Node.js/Express إنشاء مشروع
25. Frontend للـ React/Vite إنشاء مشروع
26. إعداد MongoDB Atlas + Mongoose
27. JWT نظام التسجيل والدخول بـ + User Schema بناء
28. للملفات Cloudinary إعداد
29. إعدادات الأمان الأساسية CORS بناء

المرحلة الثانية: الوظائف الأساسية (الأسبوع 3-5)

الأهداف

بناء نظام الفرص الوظيفية بالكامل مع التقديم والفلتر.

30. Job CRUD APIs بناء
31. status pending نظام نشر الفرص مع
32. Application APIs بناء
33. Multer + Cloudinary مع CV نظام رفع
34. واجهة عرض الفرص مع الفلتر والبحث
35. صفحة تفاصيل الفرصة + نموذج التقديم

المرحلة الثالثة: لوحات التحكم (الأسبوع 6-7)

الأهداف

لكل نوع مستخدم. Dashboard بناء

36. Seeker Dashboard: تقديماتي + محفوظاتي + ملفي الشخصي
37. Employer Dashboard: فرصتي + المتقدمون + الإحصائيات
38. Admin Dashboard: مراجعة الفرص + إدارة المستخدمين

المرحلة الرابعة: الإشعارات والتحسينات (الأسبوع 8)

الأهداف

العام. UX للإشعارات الفورية، وتحسين Socket.IO إضافة

- 39. دمج Socket.IO في Backend
- 40. بناء Notification Component في Frontend
- 41. لإشعارات البريد الإلكتروني Nodemailer إضافة
- 42. Responsive Design وإضافة الـ UI تحسين الـ
- 43. اختبار شامل وإصلاح الأخطاء

المرحلة الخامسة: النشر والإطلاق (الأسبوع 9)

الأهداف

نشر المشروع على الإنترنت وجعله متاحاً للمستخدمين.

- 44. Render أو Railway على Backend نشر
- 45. Netlify أو Vercel على Frontend نشر
- 46. في الإنتاج (Environment Variables) إعداد متغيرات البيئة
- 47. اختبار النظام في بيئة الإنتاج
- 48. أولي Admin إنشاء مستخدم

API أمثلة على بيانات 9.

9.1 نموذج طلب نشر فرصة

```
POST /api/jobs
Authorization: Bearer <employer_token>

{
  "title": "مصمم جرافيك للعمل الحر",
  "description": "نبحث عن مصمم جرافيك متمكن لتصميم هوية بصرية لشركة ناشئة",
  "requirements": "portfolio، تقديم، Photoshop و Illustrator خبرة سنتين، إتقان",
  "type": "online",
  "category": "تصميم جرافيك",
  "contactInfo": "059XXXXXXX: التواصل عبر الواتساب",
  "deadline": "2026-03-01"
}
```

9.2 API نموذج استجابة الـ

```
{
  "success": true,
  "message": "تم نشر الفرصة بنجاح، في انتظار موافقة الإدارة",
  "data": {
    "_id": "64f3a2b1c8e4d500123abc45",
    "title": "مصمم جرافيك للعمل الحر",
    "status": "pending",
    "employer": { "name": "أحمد محمد", "_id": "..."},
    "createdAt": "2026-01-15T10:30:00.000Z"
  }
}
```

9.3 (.env) متغيرات البيئة

```
# Server
PORT=5000
NODE_ENV=development

# Database
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/fursa

# JWT
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRE=7d

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

```
# Email
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=fursa.platform@gmail.com
EMAIL_PASS=your_app_password

# Frontend URL (for CORS)
CLIENT_URL=http://localhost:5173
```

10. فئات التخصصات المدعومة

يدعم الموقع الفئات التالية ابتداءً، مع إمكانية التوسع لاحقاً:

برمجة وتطوير	تطوير ويب، تطبيقات موبايل، برمجة عامة
تصميم	موشن جرافيك، فيديو، UI/UX، جرافيك
كتابة وترجمة	كتابة محتوى، ترجمة، تحرير
تسويق رقمي	إعلانات، SEO، إدارة سوشيال ميديا
تعليم وتدريب	تدريس خصوصي، تدريب مهني، دورات
بيانات ومحاسبة	محاسبة، تحليل بيانات، Excel
خدمة عملاء	دعم فني، رد على استفسارات
عمل ميداني	توصيل، مبيعات، تصوير، إلخ
أخرى	أي تخصص لا يندرج ضمن القائمة

اعتبارات خاصة ببيئة غزة 11.

ملاحظة مهمة

تم تصميم المنصة مع مراعاة الظروف الخاصة لقطاع غزة، بما فيها انقطاع الإنترنت وضعف الاتصال في بعض الأوقات.

11.1 تحسينات الأداء

- ضغط الصور والملفات قبل الرفع للتخفيف من حجم البيانات
- Lazy Loading للصفحات والمكونات
- للبيانات التي لا تتغير كثيراً (Caching) تخزين مؤقت
- لأن معظم المستخدمين يصلون عبر الهاتف Mobile-First تصميم
- (PWA) جزئي لعرض الفرص المحفوظة بدون إنترنت Offline دعم وضع

11.2 إمكانية الوصول

- RTL الموقع باللغة العربية بالكامل مع دعم
- الخطوط واضحة وبأحجام مناسبة
- ألوان تتوافق مع معايير إمكانية الوصول
- (3G) يعمل على شبكات بطيئة

11.3 طرق التواصل

نظراً لعدم وجود بوابات دفع إلكتروني موثوقة، تعتمد المنصة على:

- رقم واتساب أو تيليجرام في معلومات التواصل
- البريد الإلكتروني
- الاجتماع الميداني للوظائف الميدانية

12. دليل البدء السريع (Quick Start)

12.1 متطلبات التشغيل

Node.js	الإصدار 18 أو أحدث
npm	الإصدار 9 أو أحدث
MongoDB	سحابي (أو محلي) Atlas
Git	أي إصدار حديث
Clouinary	حساب مجاني

12.2 خطوات التشغيل

```
# 1. استنساخ المشروع
git clone https://github.com/youruser/fursa-platform.git

# 2. تشغيل الـ Backend
cd fursa-backend
npm install
cp .env.example .env      # وعدّل القيم
npm run dev               # يعمل على المنفذ 5000

# 3. تشغيل الـ Frontend
cd ../fursa-frontend
npm install
npm run dev               # يعمل على المنفذ 5173

# 4. الوصول إلى التطبيق
# http://localhost:5173
```